

开发任务五操作指南

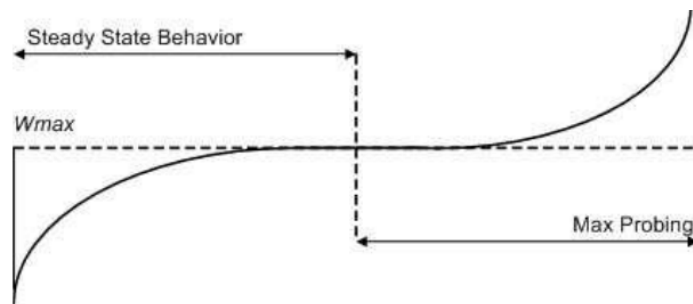
有了简单的AIMD算法实现的经验后你可以实现你想要的拥塞控制算法。在计算机网络中，拥塞控制是一项关键的技术，用于维持网络性能并避免拥塞。在给定的代码文件中，我们可以通过在 `Controller` 类中实现自定义的拥塞控制算法，以优化网络的性能。

你也可以借鉴一些经典拥塞控制算法的核心思想：

1. Cubic

背景：

Cubic算法是由Google提出的一种拥塞控制算法，旨在提高TCP在高带宽长距离网络中的性能。Cubic通过调整拥塞窗口的增长速率来适应网络状况，具有较为平滑的窗口增长曲线。



cubic窗口增长曲线

特点：

- 使用拟合立方函数来估计网络状态，根据网络状态调整窗口大小。
- 在网络容量逐渐增大时，窗口大小平滑增长。

2. Copa

背景：

Copa是一种拥塞控制算法，旨在提高TCP在高丢包率网络中的性能。它通过维护紧凑的拥塞窗口动态调整机制，减小网络对窗口大小变化的敏感性。

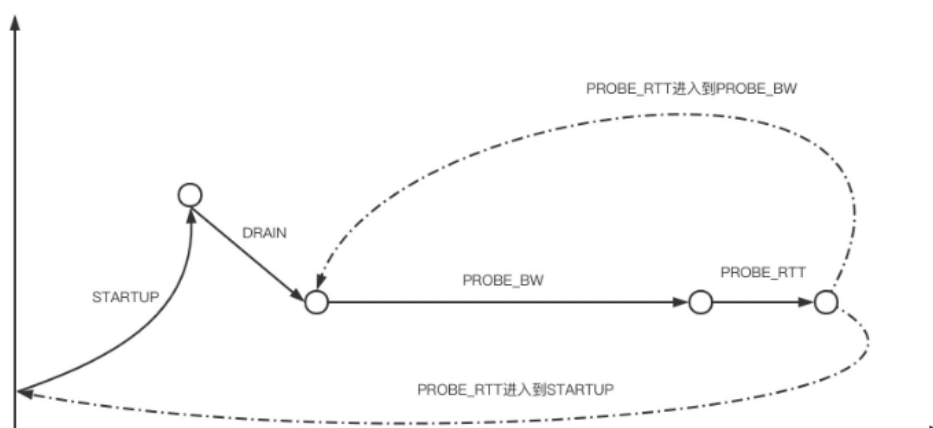
特点：

- 使用周期性的超时探测机制，以捕捉网络瞬时状态。
- 根据效用函数计算目标发送速率并调整真实发送速率接近目标发送速率。

3. BBR (Bottleneck Bandwidth and RTT)

背景：

BBR是由Google设计的一种拥塞控制算法，旨在通过动态地估计带宽和RTT（往返时间）来优化网络性能。BBR在高丢包率和高带宽环境中表现良好，能够更有效地利用网络资源。



BBR状态图

特点：

- 通过Bottleneck带宽和RTT的估计，动态调整发送窗口大小。
- 具有优秀的网络适应性，能够在不同网络条件下实现高效的拥塞控制。

4. Vegas

背景：

Vegas是一种拥塞控制算法，旨在通过主动检测网络拥塞的早期迹象来避免拥塞的发生。它通过监测丢包和往返时间等指标来调整窗口大小，以更为灵敏地适应网络状态变化。

特点：

- 主动检测网络拥塞的早期迹象，避免等待丢包的发生。
- 通过综合考虑网络信息，实现更为精准的拥塞控制。

这些拥塞控制算法在不同网络环境下有着各自的优势和特点。Cubic关注在高带宽长距离网络的平滑窗口增长，Copa专注于在高丢包率环境中维持紧凑的窗口动态调整，BBR通过综合考虑带宽和RTT实现了更为灵活的网络适应性，而Vegas则着眼于早期拥塞的检测。

开发任务：

请你修改 `controller.cc` 来实现你自己的拥塞控制算法并命名为 `xxx.cc`，记得给你的算法起个独特的名字！

提交要求：

提交你自己设计的 `xxx.cc` 代码以及设计说明书一份。代码要求一人一码，严禁抄袭！

设计说明书要求：

要求写清楚任务目标、任务实施过程中的必要细节，包括但不限于算法细节描述、机制原理说明和过程结果截图等。